Intermediate (multi-agent content pipeline design) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `multi_agent_pipeline_design.md`: Project 4's shape, designed fresh from a request instead of followed from a guide — phases, pattern, state, and what happens when a check fails. **If not:** you'd only ever have built multi-agent systems someone else already designed.

Write your own design first, in `practice/multi_agent_pipeline_design.md`.

The request: "research a topic, draft an article, fact-check it, only publish if fact-check passes."

Solution

Story: a loop without a limit, and a single "try again" for every kind of failure, are the two places this pipeline breaks for real. A cap, an escalation path, and two different loop-back targets fix both. **If not:** some articles would loop forever, and missing research would be sent to the writer to "fix" by rewording.

Same three agents, hardened where an unbounded loop and an underspecified state object would actually fail:

- **A hard revision cap with a named escalation path.** `MAX_REVISIONS = 3`. If fact-check still fails after 3 writer revisions, the article escalates to a human editor with the full history (every draft, every round of fact-check notes) — it does not loop again. This is a direct application of Doc14's failure vocabulary: an unbounded retry loop is a real failure mode, not just a hypothetical one, and this document's "Expected Behavior" explicitly expects you to ask "what stops this from looping forever" before being asked.
- **Two different loop-back destinations, not one.** If the fact-checker finds a claim with *no supporting source at all*, that's a research gap — it routes back to the researcher, not the writer, because no amount of rewriting fixes missing information. If the fact-checker finds a claim that's source-backed but overstated or misworded, that routes back to the writer with specific notes. Collapsing these into one generic "try again" loses the distinction that actually determines whether the retry can succeed.
- **A shared state object with a revision counter and structured notes**, not just a draft passed hand to hand — every agent downstream needs to see *why* the last attempt was rejected, or the writer is revising blind and likely repeating the same mistake.

- **Monitoring on revision_count and route taken**, so articles that consistently hit the cap become a visible signal that a specific topic or source set is the problem — not just individual bot failures nobody's tracking in aggregate.
- **The fact-checker flags, it doesn't rewrite** — named as a deliberate trade-off, not an unquestioned default: at the scale this exercise describes, the extra round-trip cost of sending it back to the writer is worth the cleaner separation of concerns and the auditability of "the fact-checker's only output is a pass/fail plus notes." At very high volume, this is exactly the kind of choice worth revisiting.

Why this is the stronger design, and what the weaker one gets wrong: The Basic version's "try again" loop has no answer for what happens when fact-check simply never passes — which is not an edge case, it's a certainty for some articles (a genuinely contradictory topic, a core claim the writer keeps reintroducing). Without a cap, that's an infinite loop burning tokens and never resolving; a reviewer will ask about this immediately, and "it just keeps retrying" is not an acceptable answer. It also treats every fact-check failure the same way, which means a missing-research problem gets sent to the writer to fix by rewording — something rewording can't actually fix. The stronger design isn't adding complexity for its own sake: the cap, the escalation path, and the two loop-back destinations each trace directly to a specific way the simple loop breaks.

Which one should you actually present in an interview? Present the Basic version's shape first — it's the correct instinct on phases and the loop-back concept. Then, unprompted, raise "what happens if fact-check never passes" and walk through the cap and escalation path yourself — that's the exact behavior Doc16 expects, and it's usually the question that decides whether the interviewer sees you as someone who's actually thought about failure, not just the happy path.